LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK
“Abstraction”



disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

AFDHAL DINILHAK, M.Kom.

Asisten Praktikum:

SITI FADHILAH RAHMI

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman berorientasi objek (Object-Oriented Programming atau OOP), penyusunan program yang baik perlu memperhatikan kemudahan dalam pengembangan dan pemeliharaan kode. Semakin besar suatu aplikasi, semakin banyak pula class dan fungsi yang harus dikelola. Oleh karena itu, diperlukan konsep yang dapat membantu menyederhanakan struktur program, salah satunya adalah abstraksi

Abstraksi merupakan konsep dalam OOP yang digunakan untuk menyembunyikan bagian implementasi yang tidak perlu ditampilkan dan hanya memperlihatkan fungsi utama dari suatu objek. Dalam Java, abstraksi dapat diterapkan menggunakan abstract class dan interface. Keduanya memiliki fungsi yang berbeda, tetapi sama-sama membantu dalam mengatur struktur dan perilaku program.

Selain itu, Java juga menyediakan beberapa fitur pendukung, seperti default method, static method, private method, modifier final, serta penggunaan nested class dan anonymous class. Fitur-fitur tersebut dapat digunakan untuk membuat program lebih terstruktur dan memudahkan penambahan fitur baru tanpa harus mengubah banyak bagian kode yang sudah ada.

Melalui praktikum ini, mahasiswa mempelajari cara menerapkan konsep abstraksi pada program Java, mulai dari membuat rancangan class, menulis kode program, hingga memeriksa dan memperbaiki kesalahan desain. Dengan demikian, mahasiswa dapat memahami penggunaan abstraksi secara langsung melalui proyek yang dikerjakan selama praktikum.

1.2 Tujuan Praktikum

1. Menjelaskan konsep abstraction, termasuk perbedaan abstract class dan interface serta kapan masing-masing tepat dipakai.
2. Mengimplementasikan abstraction di Java: abstract class dan method, interface beserta default/static/private method, interface inheritance, final, serta static nested, inner, dan anonymous class.

3. Merancang program yang mudah diperluas dengan prinsip program to an interface, termasuk menambah fitur ke interface tanpa merusak kode lama dan menggambarkannya dalam class diagram UML.
4. Mengevaluasi desain kode: menemukan dan memperbaiki masalah abstraction pada kode yang ada, serta memberi alasan atas pilihan desain yang dibuat.
5. Membangun dan menguji proyek Java secara mandiri di VS Code, termasuk membaca pesan error compiler dan hasil pemeriksaan otomatis untuk memperbaiki kode sendiri.

1.3 Manfaat Praktikum

1. Praktikum ini membantu mahasiswa memahami penerapan abstraksi dalam Java, terutama dalam menyembunyikan detail program dan membagi fungsi setiap class sesuai tanggung jawabnya.
2. Mahasiswa dapat belajar menyusun class dan interface dengan lebih teratur sehingga program lebih mudah dipahami, dikembangkan, dan diperbaiki.
3. Mahasiswa memperoleh pengalaman dalam menemukan kesalahan atau kekurangan pada desain program, kemudian memperbaikinya melalui proses refactoring.
4. Mahasiswa dapat mempraktikkan langsung pembuatan dan pengujian program Java, mulai dari menyusun kode hingga menjalankan pemeriksaan untuk memastikan program berjalan sesuai ketentuan.

BAB II
PEMBAHASAN

2.1 Latihan Praktikum

2.1.1 L1 – Abstract class dan abstract method

a. L1.a Product

```
package shop.model;

public abstract class Product {
    private final String sku;
    private final String name;
    private final long price; // in rupiah

    protected Product(String sku, String name, long
price) {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU must
not be blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must
not be blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price
must not be negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public String getSku() { return sku; }
```

```
    public String getName() { return name; }
    public long getPrice() { return price; }

    /** Price the customer actually pays. Subclass may
override it. */
    public long getFinalPrice() { return getPrice(); }

    /** Every concrete product must state its category.
*/
    public abstract String category();

    /** A concrete method that relies on the abstract
method category(). */
    public String getInfo() {
        return "[" + category() + "] " + name + " - Rp" +
getFinalPrice();
    }
}
```

Merupakan abstract class Product yang mempunyai constructor yang menolak SKU kosong, nama kosong, atau harga negatif, serta mendefinisikan abstract method category() sebagai method wajib bagi seluruh subclass nya agar menyatakan kategorinya sendiri.

b. L1.b Class PhysicalProduct

```
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements
Shippable {
    private final int weightGrams; // in kg

    public PhysicalProduct(String sku, String name, long
price, int weightGrams) {
```

```
        super(sku, name, price);
        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight
must be positive");
        }
        this.weightGrams = weightGrams;
    }

    @Override
    public int getWeightGrams() { return weightGrams; }

    @Override
    public String category() {
        return "Fisik";
    }

}
```

Merupakan subclass dari Product. Class ini mempresentasikan validasi berat dalam gram (weightGrams > 0) serta meng-override method category() untuk mengembalikan String “Fisik”.

c. L1.c Class DigitalProduct

```
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements
Discountable {
    private final String downloadUrl;
    private static final int DIGITAL_DISCOUNT_PERCENT =
10;

    public DigitalProduct(String sku, String name, long
price, String downloadUrl) {
```

```
        super(sku, name, price);
        if (downloadUrl == null || downloadUrl.isBlank())
{
            throw new IllegalArgumentException("Download
URL must not be blank");
        }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() {
        return downloadUrl;
    }

    @Override
    public String category() {
        return "Digital";
    }

    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }
}
```

Merupakan subclass dari Product. Class ini menmbahkan atribut khusus berupa URL unduhan (downloadURL) yang divalidasi agar tidak kosong dan meng-override method category() untuk mengembalikan String “Digital”.

d. L1.d Class ShippingService

```java
package shop.shipping;

public abstract class ShippingService {
    private final String name;
    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must
not be blank");
        }
        this.name = name;
    }

    public String getName() { return name; }

    /** Rate in rupiah per started kilogram */
    protected abstract long ratePerKg();

    /** Weight is rounded up: 700 g -> 1 kg, 1001 g -> 2
kg */
    public final long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
           return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000; //
round up to nearest kg
        return kilograms * ratePerKg();
    }

}
```

Merupakan abstract class untuk pengiriman yang menggunakan rumus perhitungan biaya kirim Bersama (pembulatan berat per kilogram ke atas), sementara tarif per kilo nya diserahkan ke masing-masing kurir melalui abstract method ratePerKg().

e. L1.e Class RegularShipping dan ExpressShipping

```
package shop.shipping;

public class RegularShipping extends ShippingService {
    public RegularShipping() {
        super("Reguler");
    }

    @Override
    protected long ratePerKg() {
        return 10_000;
    }

}

package shop.shipping;

public class ExpressShipping extends ShippingService {
    public ExpressShipping() {
        super("Ekspres");
    }

    @Override
    protected long ratePerKg() {
        return 18_000;
    }

}
```

RegularShipping adalah implementasi kurir regular dengan tarif tetap, sedangkan ExpressShipping untuk menerapkan tarif kurir ekspress khusus (Rp18.000 per kg) dengan nama kurir “Ekspress”.

2.1.2 L2 – Interface dan program to an interface

a. L2.a Shippable

```java
package shop.shipping;

/** Capability: anything that has weight and can be
shipped. */
public interface Shippable {
    int getWeightGrams();
}
```

Mendefinisikan kemampuan bagi objek apapun yang memiliki berat dan dapat kiriman (getWeightGrams()). Class PhysicalProduct memakai ini, sementara DigitalProduct tidak.

b. L2.b Interface PaymentMethod

```java
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    default long adminFee(long amount) {
        return 0;
    }

     /** @return true if the payment success. */
    boolean pay(long amount);
}
```

Kode antarmuka umum untuk metode pembayaran yang mendefinisikan method getName() dan pay(long amount).

c. L2.c Class EWalletPayment

```java
package shop.payment;
```

```
public class EWalletPayment implements RefundablePayment
{
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance)
{
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException("Provider
must not be blank");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Balance
must not be negative");
        }
        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }

    public long getBalance() {
        return balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
```

```
        balance -= amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }
}
```

Class yang mengimplementasikan interface PaymentMethod. Class ini menyediakan method abstract getName() untuk mengembalikan nama layanan pembayaran dan pay(long amount) untuk memproses pemotongan saldo dompet digital.

d. L2.d Class BankTransferMethod

```
package shop.payment;

public class BankTransferPayment implements PaymentMethod
{
    private String bankName;

    public BankTransferPayment(String bankName) {
        if (bankName == null || bankName.isBlank()) {
            throw new IllegalArgumentException("Bank name
must not be blank");
        }
        this.bankName = bankName;
    }

    @Override
```

```
    public String getName() {
        return "Transfer " + bankName;
    }

    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }

    @Override
    public long adminFee(long amount) {
        return shop.StudentInfo.TRANSFER_ADMIN_FEE;
    }
}
```

Class ini juga mengimplementasikan PaymentMethod. Class ini menjelaskan cara kerja transaksi pembayaran via transfer bank, method pay() memproses pembayaran langsung melalui transfer rekening bank tanpa memerlukan validasi saldo dompet digital yang rumit.

e. L2.e Class CheckoutService

```
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment; // <--- Jangan
lupa import ini!
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {
    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
```

```
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) {
// a capability, not a class
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items,
ShippingService shipping) {
        return subtotal(items) +
shipping.calculateCost(totalWeight(items));
    }

    public boolean checkout(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
            return false;
        }
        long finalAmount = grandTotal(items, shipping,
payment);
        return payment.pay(finalAmount);
    }
```

```
    public boolean refund(RefundablePayment payment, long
amount) {
        return payment.refund(amount);
    }

    public long grandTotal(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        long baseTotal = total(items, shipping);
        return baseTotal + payment.adminFee(baseTotal);
    }

}
```

Class ini mengimplementasikan prinsip program to an interface, di mana layanan checkout hanya bergantung pada abstraksi interface seperti PaymentMethod dan Shippable.

f. L2.f ShopApp

```
package shop.app;

import java.util.List;
import shop.model.Address;
import shop.model.DigitalProduct;
import shop.model.Order;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.EWalletPayment;
import shop.payment.PaymentMethod;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;
```

```java
public class ShopApp {
    public static void main(String[] args) {
        // L1: abstract class and abstract method
        Product coffee = new PhysicalProduct("KOPI-001",
"Kopi Arabika Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-002",
"Rendang Kemasan 250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003", "E-
book Resep Kue Nusantara", 50_000,
                "https://toko-umkm.example/unduh/ebk-
003");

        List<Product> cart = List.of(coffee, rendang,
ebook);
        System.out.println("== Katalog ==");
        for (Product product : cart) {
            System.out.println(product.getInfo());
        }

        // L2: interface and program to an interface
        CheckoutService checkout = new CheckoutService();
        ShippingService regular = new RegularShipping();
        ShippingService express = new ExpressShipping();
        System.out.println("\n== Ringkasan ==");
        System.out.println("Subtotal      : Rp" +
checkout.subtotal(cart));
        System.out.println("Berat kiriman : " +
checkout.totalWeight(cart) + " g");
        System.out.println("Total Reguler : Rp" +
checkout.total(cart, regular));
        System.out.println("Total Ekspres : Rp" +
checkout.total(cart, express));
```

```
        // L3: interface inheritance, multiple
interfaces, default method
        EWalletPayment wallet = new
EWalletPayment("Dompetku", 300_000);
        PaymentMethod transfer = new
BankTransferPayment("Bank Nagari");

        System.out.println("\n== Pembayaran ==");
        System.out.println(wallet.getName() + " berhasil?
" + checkout.checkout(cart, regular, wallet));
        System.out.println("Sisa saldo: Rp" +
wallet.getBalance());
        System.out.println(transfer.getName() + "
berhasil? " + checkout.checkout(cart, express,
transfer));

        CreditCardPayment card = new
CreditCardPayment("Siti Rahma", 1_000_000);
        System.out.println(card.getName() + " berhasil? "
+ checkout.checkout(cart, express, card));
        System.out.println("Cicilan 3 bulan: Rp" +
card.monthlyInstallment(checkout.total(cart, express),
3));

        System.out.println("\n== Refund e-book ==");
        System.out.println("Refund e-wallet berhasil? " +
checkout.refund(wallet, ebook.getFinalPrice()));
        System.out.println("Saldo setelah refund: Rp" +
wallet.getBalance());

        // checkout.refund(transfer, 1_000); // compile
error: BankTransferPayment is not RefundablePayment
```

```
        // L5.c: Static nested, inner, dan anonymous
class
        Address address = new Address("Jl. Moh Hatta No.
10", "Padang", "25121");
        Order order = new Order("ORD-0001", address);
        order.addLine(coffee, 2);
        order.addLine(ebook, 1);

        Order.Receipt receipt = order.new Receipt(); //
inner object needs an outer object
        System.out.println("\n" + receipt.toText());

        PaymentMethod voucher = new PaymentMethod() { //
anonymous class
            private long remaining = 50_000;
            @Override
            public String getName() { return "Voucher
Promo Hari Ini"; }
            @Override
            public boolean pay(long amount) {
                if (amount <= 0 || amount > remaining) {
                    return false;
                }
                remaining -= amount;
                return true;
            }
        };

        System.out.println("\n" + voucher.getName() + "
untuk e-book berhasil? "
            + checkout.checkout(List.of(ebook), regular,
voucher));
    }
```

```
}
```

Driver class untuk menguji katalog produk, menghitung subtotal keranjang, total berat kiriman, dan perbandingan biaya pengiriman.

2.1.3 L3 – Interface lanjutan: default, static, private method dan interface inheritance

a. L3.a Class Discountable

```
package shop.discount;

public interface Discountable {
    /** Abstract method: setiap kelas konkret menentukan persentase diskonnya */
    int discountPercent();

    /** Default method: menghitung harga akhir setelah dipotong diskon */
    default long applyDiscount(long price) {
        return price - percentOf(price, safePercent());
    }

    /** Static method: utilitas pembantu untuk menghitung sekian persen dari suatu jumlah */
    static long percentOf(long amount, int percent) {
        return amount * percent / 100;
    }

    /** Private method: memvalidasi agar persentase diskon selalu berada di rentang 0 - 100 */
    private int safePercent() {
        int percent = discountPercent();
        if (percent < 0) {
            return 0;
        }
```

```
        return Math.min(percent, 100);
    }
}
```

Memuat abstract method (discountPercent()), default method (applyDiscount()) untuk menghitung harga setelah diskon, static method (percentOf()), serta private helper method (safePercent()).

b. L3.b Class DigitalProduct

```
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements
Discountable {
    private final String downloadUrl;
    private static final int DIGITAL_DISCOUNT_PERCENT =
10;

    public DigitalProduct(String sku, String name, long
price, String downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null || downloadUrl.isBlank())
{
            throw new IllegalArgumentException("Download
URL must not be blank");
        }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() {
        return downloadUrl;
    }

    @Override
```

```
    public String category() {
        return "Digital";
    }

    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }
}
```

Menerapkan interface Discountable pada produk digital dengan memberikan diskon tetap sebesar 10%, dan meng-override getFinalPrice() agar mengembalikan applyDiscount(getPrice()). Harga e-book 50000 menjadi 45000.

c. L3.c Interface inheritance

```
package shop.payment;

public interface RefundablePayment extends PaymentMethod
{
    boolean refund(long amount);
    default String policy() {
        return "Refund maksimal 7 hari";
    }

}
package shop.payment;

public interface InstallmentPayment extends PaymentMethod
{
```

```
    int maxInstallmentMonths();

    /** Monthly amount, rounded up. */
    default long monthlyInstallment(long amount, int
months) {
        if (months < 1 || months >
maxInstallmentMonths()) {
            throw new IllegalArgumentException(
                "Months must be between 1 and " +
maxInstallmentMonths());
        }
        return (amount + months - 1) / months;
    }

    default String policy() {
        return "Cicilan maksimal " +
maxInstallmentMonths() + " bulan";
    }
}
```

Interface mewarisi interface lain menggunakan extends untuk menambahkan kemampuan refund dan cicilan.

d. L3.d EWalletPayment, CreditCardPayment, CheckoutService, dan ShopApp

1. EWalletPayment

```
package shop.payment;

public class EWalletPayment implements
RefundablePayment {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long
balance) {
```

```
        if (provider == null || provider.isBlank()) {
            throw new
IllegalArgumentException("Provider must not be blank");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Balance
must not be negative");
        }
        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }

    public long getBalance() {
        return balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
```

```
            return false;
        }
        balance += amount;
        return true;
    }
}
```

Mengubah EWalletPayment menjadi implements RefundablePayment. Method refund(amount) mengembalikan false jika amount <= 0, dan mengembalikan true serta menambah saldo jika berhasil.

2. CreditCardPayment

```
package shop.payment;

public class CreditCardPayment implements
RefundablePayment, InstallmentPayment {
    private final String cardHolder;
    private final long creditLimit;
    private long usedCredit;

    public CreditCardPayment(String cardHolder, long
creditLimit) {
        if (cardHolder == null || cardHolder.isBlank())
{
            throw new IllegalArgumentException("Card
holder must not be blank");
        }
        if (creditLimit < 0) {
            throw new IllegalArgumentException("Credit
limit must not be negative");
        }
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
        this.usedCredit = 0;
    }
```

```
    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }

    public long getUsedCredit() {
        return usedCredit;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || usedCredit + amount >
creditLimit) {
            return false;
        }
        usedCredit += amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;
        }
        usedCredit -= amount;
        return true;
    }

    @Override
    public int maxInstallmentMonths() {
        return 12;
    }
```

```
    @Override
    public long adminFee(long amount) {
        return  amount *
shop.StudentInfo.CARD_ADMIN_PERCENT / 100;
    }

    @Override
    public String policy() {
        return RefundablePayment.super.policy() + "; "
+ InstallmentPayment.super.policy();
    }
}
```

Class yang mengimplementasikan 2 interface, dilengkapi validasi limit kredit saat pembayaran (pay), validasi pengurangan utang saat pengembalian dana (refund), serta penetapan batas cicilan maksimal 12 bulan.

3. CheckoutService

```
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment; // <--- Jangan
lupa import ini!
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {
    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
```

```
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable)
{ // a capability, not a class
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items,
ShippingService shipping) {
        return subtotal(items) +
shipping.calculateCost(totalWeight(items));
    }

    public boolean checkout(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
            return false;
        }
        long finalAmount = grandTotal(items, shipping,
payment);
        return payment.pay(finalAmount);
    }

    public boolean refund(RefundablePayment payment,
long amount) {
```

```
        return payment.refund(amount);
    }

    public long grandTotal(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        long baseTotal = total(items, shipping);
        return baseTotal + payment.adminFee(baseTotal);
    }

}
```

Menambahkan method baru di CheckoutService yang hanya menerima parameter bertipe RefundablePayment sehingga sistem aman dari kesalahan tipe.

4. ShopApp

```
package shop.app;

import java.util.List;
import shop.model.Address;
import shop.model.DigitalProduct;
import shop.model.Order;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.EWalletPayment;
import shop.payment.PaymentMethod;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;

public class ShopApp {
    public static void main(String[] args) {
```

```
        // L1: abstract class and abstract method
        Product coffee = new PhysicalProduct("KOPI-
001", "Kopi Arabika Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-
002", "Rendang Kemasan 250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003",
"E-book Resep Kue Nusantara", 50_000,
                "https://toko-umkm.example/unduh/ebk-
003");

        List<Product> cart = List.of(coffee, rendang,
ebook);
        System.out.println("== Katalog ==");
        for (Product product : cart) {
            System.out.println(product.getInfo());
        }

        // L2: interface and program to an interface
        CheckoutService checkout = new
CheckoutService();
        ShippingService regular = new
RegularShipping();
        ShippingService express = new
ExpressShipping();
        System.out.println("\n== Ringkasan ==");
        System.out.println("Subtotal      : Rp" +
checkout.subtotal(cart));
        System.out.println("Berat kiriman : " +
checkout.totalWeight(cart) + " g");
        System.out.println("Total Reguler : Rp" +
checkout.total(cart, regular));
        System.out.println("Total Ekspres : Rp" +
checkout.total(cart, express));
```

```
        // L3: interface inheritance, multiple
interfaces, default method
        EWalletPayment wallet = new
EWalletPayment("Dompetku", 300_000);
        PaymentMethod transfer = new
BankTransferPayment("Bank Nagari");

        System.out.println("\n== Pembayaran ==");
        System.out.println(wallet.getName() + "
berhasil? " + checkout.checkout(cart, regular,
wallet));
        System.out.println("Sisa saldo: Rp" +
wallet.getBalance());
        System.out.println(transfer.getName() + "
berhasil? " + checkout.checkout(cart, express,
transfer));

        CreditCardPayment card = new
CreditCardPayment("Siti Rahma", 1_000_000);
        System.out.println(card.getName() + " berhasil?
" + checkout.checkout(cart, express, card));
        System.out.println("Cicilan 3 bulan: Rp" +
card.monthlyInstallment(checkout.total(cart, express),
3));

        System.out.println("\n== Refund e-book ==");
        System.out.println("Refund e-wallet berhasil? "
+ checkout.refund(wallet, ebook.getFinalPrice()));
        System.out.println("Saldo setelah refund: Rp" +
wallet.getBalance());
```

```
        // checkout.refund(transfer, 1_000); // compile
error: BankTransferPayment is not RefundablePayment

        // L5.c: Static nested, inner, dan anonymous
class
        Address address = new Address("Jl. Moh Hatta
No. 10", "Padang", "25121");
        Order order = new Order("ORD-0001", address);
        order.addLine(coffee, 2);
        order.addLine(ebook, 1);

        Order.Receipt receipt = order.new Receipt(); //
inner object needs an outer object
        System.out.println("\n" + receipt.toText());

        PaymentMethod voucher = new PaymentMethod() {
// anonymous class
            private long remaining = 50_000;
            @Override
            public String getName() { return "Voucher
Promo Hari Ini"; }
            @Override
            public boolean pay(long amount) {
                if (amount <= 0 || amount > remaining)
{
                    return false;
                }
                remaining -= amount;
                return true;
            }
        };
```

```
        System.out.println("\n" + voucher.getName() + "
untuk e-book berhasil? "
                + checkout.checkout(List.of(ebook),
regular, voucher));
    }
}
```

Mencoba melakukan pengembalian dana menggunakan BankTransferPayment (checkout.refund(new BankTransferPayment("Bank Nagari"), 1_000);). Kompiler Java akan menolak baris ini karena BankTransferPayment tidak mengimplementasikan RefundablePayment, membuktikan kekuatan sistem tipe statis Java tanpa perlu pengecekan if manual.

e. L3.e Lengkapi ShopApp

```
package shop.app;

import java.util.List;
import shop.model.Address;
import shop.model.DigitalProduct;
import shop.model.Order;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.EWalletPayment;
import shop.payment.PaymentMethod;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;

public class ShopApp {
    public static void main(String[] args) {
        // L1: abstract class and abstract method
```

```
        Product coffee = new PhysicalProduct("KOPI-001",
"Kopi Arabika Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-002",
"Rendang Kemasan 250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003", "E-
book Resep Kue Nusantara", 50_000,
                "https://toko-umkm.example/unduh/ebk-
003");

        List<Product> cart = List.of(coffee, rendang,
ebook);
        System.out.println("== Katalog ==");
        for (Product product : cart) {
            System.out.println(product.getInfo());
        }

        // L2: interface and program to an interface
        CheckoutService checkout = new CheckoutService();
        ShippingService regular = new RegularShipping();
        ShippingService express = new ExpressShipping();
        System.out.println("\n== Ringkasan ==");
        System.out.println("Subtotal      : Rp" +
checkout.subtotal(cart));
        System.out.println("Berat kiriman : " +
checkout.totalWeight(cart) + " g");
        System.out.println("Total Reguler : Rp" +
checkout.total(cart, regular));
        System.out.println("Total Ekspres : Rp" +
checkout.total(cart, express));

        // L3: interface inheritance, multiple
interfaces, default method
```

```
        EWalletPayment wallet = new
EWalletPayment("Dompetku", 300_000);
        PaymentMethod transfer = new
BankTransferPayment("Bank Nagari");

        System.out.println("\n== Pembayaran ==");
        System.out.println(wallet.getName() + " berhasil?
" + checkout.checkout(cart, regular, wallet));
        System.out.println("Sisa saldo: Rp" +
wallet.getBalance());
        System.out.println(transfer.getName() + "
berhasil? " + checkout.checkout(cart, express,
transfer));

        CreditCardPayment card = new
CreditCardPayment("Siti Rahma", 1_000_000);
        System.out.println(card.getName() + " berhasil? "
+ checkout.checkout(cart, express, card));
        System.out.println("Cicilan 3 bulan: Rp" +
card.monthlyInstallment(checkout.total(cart, express),
3));

        System.out.println("\n== Refund e-book ==");
        System.out.println("Refund e-wallet berhasil? " +
checkout.refund(wallet, ebook.getFinalPrice()));
        System.out.println("Saldo setelah refund: Rp" +
wallet.getBalance());

        // checkout.refund(transfer, 1_000); // compile
error: BankTransferPayment is not RefundablePayment

        // L5.c: Static nested, inner, dan anonymous
class
```

```
        Address address = new Address("Jl. Moh Hatta No.
10", "Padang", "25121");
        Order order = new Order("ORD-0001", address);
        order.addLine(coffee, 2);
        order.addLine(ebook, 1);

        Order.Receipt receipt = order.new Receipt(); //
inner object needs an outer object
        System.out.println("\n" + receipt.toText());

        PaymentMethod voucher = new PaymentMethod() { //
anonymous class
            private long remaining = 50_000;
            @Override
            public String getName() { return "Voucher
Promo Hari Ini"; }
            @Override
            public boolean pay(long amount) {
                if (amount <= 0 || amount > remaining) {
                    return false;
                }
                remaining -= amount;
                return true;
            }
        };

        System.out.println("\n" + voucher.getName() + "
untuk e-book berhasil? "
            + checkout.checkout(List.of(ebook), regular,
voucher));
    }
}
```

Menambahkan import untuk EWalletPayment, BankTransferPayment, CreditCardPayment, dan PaymentMethod. Menguji seluruh transaksi pembayaran, cicilan, dan simulasi pengembalian dana (*refund*) pada objek diskon.

2.1.4 L4 – final method dan final class

a. L4.a Kunci rumus ongkir

```
package shop.shipping;

public abstract class ShippingService {
    private final String name;
    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must
not be blank");
        }
        this.name = name;
    }

    public String getName() { return name; }

    /** Rate in rupiah per started kilogram */
    protected abstract long ratePerKg();

    /** Weight is rounded up: 700 g -> 1 kg, 1001 g -> 2
kg */
    public final long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
           return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000; //
round up to nearest kg
        return kilograms * ratePerKg();
```

```
    }

}
```

Membuat final method pada method calculateCost() di class induk ShippingService agar rumus pembulatan berat tidak bisa diubah oleh subclass kurir.

b. L4.b Address immutable

```
package shop.model;

public final class Address {
    private final String street;
    private final String city;
    private final String postalCode;

    public Address(String street, String city, String
postalCode) {
        if (street == null || street.isBlank()) {
            throw new IllegalArgumentException("Street
must not be blank");
        }
        if (city == null || city.isBlank()) {
            throw new IllegalArgumentException("City must
not be blank");
        }
        if (postalCode == null || postalCode.isBlank()) {
            throw new IllegalArgumentException("Postal
code must not be blank");
        }
        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
    }
```

```
    public String getStreet() {
        return street;
    }

    public String getCity() {
        return city;
    }

    public String getPostalCode() {
        return postalCode;
    }

    @Override
    public String toString() {
        return street + ", " + city + " " + postalCode;
    }
}
```

Membuat class alamat menjadi final dengan field private final tanpa setter agar objek bersifat immutable (aman dari perubahan tidak disengaja).

2.1.5 L5 – Nested class: static nested, inner, dan anonymous class

a. L5.a Class Order

```
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();
```

```
    public Order(String orderId, Address shippingAddress)
{
        if (orderId == null || orderId.isBlank() ||
shippingAddress == null) {
            throw new IllegalArgumentException("Order id
and address are required");
        }
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() {
        return orderId;
    }

    public Address getShippingAddress() {
        return shippingAddress;
    }

    public void addLine(Product product, int quantity) {
        lines.add(new Line(product, quantity));
    }

    public List<Line> getLines() {
        return Collections.unmodifiableList(lines);
    }

    // 2. Menghitung total subtotal dari semua baris
pesanan
    public long getItemsTotal() {
        long total = 0;
        for (Line line : lines) {
            total += line.subtotal();
```

```
        }
        return total;
    }

    // 4. Membuat objek Receipt (inner class)
    public Receipt createReceipt() {
        return new Receipt();
    }

    // 1. Static nested class: Line
    public static class Line {
        private final Product product;
        private final int quantity;

        public Line(Product product, int quantity) {
            if (product == null || quantity <= 0) {
                throw new
IllegalArgumentException("Product must not be null and
quantity must be positive");
            }
            this.product = product;
            this.quantity = quantity;
        }

        public Product getProduct() {
            return product;
        }

        public int getQuantity() {
            return quantity;
        }

        public long subtotal() {
```

```
            return product.getFinalPrice() * quantity;
        }
    }

    // 3. Inner class (non-static): Receipt
    public class Receipt {
        public String toText() {
            StringBuilder sb = new StringBuilder();
            sb.append("STRUK PESANAN
").append(Order.this.orderId).append("\n");
            for (Line line : Order.this.lines) {
                sb.append("- ")
                  .append(line.getProduct().getName())
                  .append(" x")
                  .append(line.getQuantity())
                  .append(" = Rp")
                  .append(line.subtotal())
                  .append("\n");
            }
            sb.append("Total barang:
Rp").append(getItemsTotal()).append("\n");
            sb.append("Kirim ke:
").append(Order.this.shippingAddress);
            return sb.toString();
        }
    }
}
```

Class Order yang menyimpan atribut pesanan (ID pesanan dan alamat pengiriman) serta daftar item barang (lines). Line sebagai static nested class. Line dirancang sebagai static karena bertindak sebagai struktur pembantu baris item belanja yang berdiri sendiri dan tidak memerlukan akses ke state objek Order di luarnya, sehingga lebih efisien secara memori.

b. L5.b tambahan Class Order

```java
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();

    public Order(String orderId, Address shippingAddress)
{
        if (orderId == null || orderId.isBlank() ||
shippingAddress == null) {
            throw new IllegalArgumentException("Order id
and address are required");
        }
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() {
        return orderId;
    }

    public Address getShippingAddress() {
        return shippingAddress;
    }

    public void addLine(Product product, int quantity) {
        lines.add(new Line(product, quantity));
    }
```

```
    public List<Line> getLines() {
        return Collections.unmodifiableList(lines);
    }

    // 2. Menghitung total subtotal dari semua baris
pesanan
    public long getItemsTotal() {
        long total = 0;
        for (Line line : lines) {
            total += line.subtotal();
        }
        return total;
    }

    // 4. Membuat objek Receipt (inner class)
    public Receipt createReceipt() {
        return new Receipt();
    }

    // 1. Static nested class: Line
    public static class Line {
        private final Product product;
        private final int quantity;

        public Line(Product product, int quantity) {
            if (product == null || quantity <= 0) {
                throw new
IllegalArgumentException("Product must not be null and
quantity must be positive");
            }
            this.product = product;
            this.quantity = quantity;
```

```
        }

        public Product getProduct() {
            return product;
        }

        public int getQuantity() {
            return quantity;
        }

        public long subtotal() {
            return product.getFinalPrice() * quantity;
        }
    }

    // 3. Inner class (non-static): Receipt
    public class Receipt {
        public String toText() {
            StringBuilder sb = new StringBuilder();
            sb.append("STRUK PESANAN
").append(Order.this.orderId).append("\n");
            for (Line line : Order.this.lines) {
                sb.append("- ")
                  .append(line.getProduct().getName())
                  .append(" x")
                  .append(line.getQuantity())
                  .append(" = Rp")
                  .append(line.subtotal())
                  .append("\n");
            }
            sb.append("Total barang:
Rp").append(getItemsTotal()).append("\n");
```

```
            sb.append("Kirim ke:
").append(Order.this.shippingAddress);
            return sb.toString();
        }
    }
}
```

Menambahkan logika perhitungan total barang (getItemsTotal()) dan Receipt sebagai inner class. Receipt butuh akses langsung ke state objek luarnya (Order.this.orderId dan Order.this.shippingAddress) untuk mencetak teks struk pesanan secara akurat. Dilengkapi juga dengan method createReceipt() untuk mengembalikan instance dari inner class tersebut.

c. L5.c Lengkapi ShopApp

```
package shop.app;

import java.util.List;
import shop.model.Address;
import shop.model.DigitalProduct;
import shop.model.Order;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.EWalletPayment;
import shop.payment.PaymentMethod;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;

public class ShopApp {
    public static void main(String[] args) {
        // L1: abstract class and abstract method
```

```
        Product coffee = new PhysicalProduct("KOPI-001",
"Kopi Arabika Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-002",
"Rendang Kemasan 250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003", "E-
book Resep Kue Nusantara", 50_000,
                "https://toko-umkm.example/unduh/ebk-
003");

        List<Product> cart = List.of(coffee, rendang,
ebook);
        System.out.println("== Katalog ==");
        for (Product product : cart) {
            System.out.println(product.getInfo());
        }

        // L2: interface and program to an interface
        CheckoutService checkout = new CheckoutService();
        ShippingService regular = new RegularShipping();
        ShippingService express = new ExpressShipping();
        System.out.println("\n== Ringkasan ==");
        System.out.println("Subtotal      : Rp" +
checkout.subtotal(cart));
        System.out.println("Berat kiriman : " +
checkout.totalWeight(cart) + " g");
        System.out.println("Total Reguler : Rp" +
checkout.total(cart, regular));
        System.out.println("Total Ekspres : Rp" +
checkout.total(cart, express));

        // L3: interface inheritance, multiple
interfaces, default method
```

```
        EWalletPayment wallet = new
EWalletPayment("Dompetku", 300_000);
        PaymentMethod transfer = new
BankTransferPayment("Bank Nagari");

        System.out.println("\n== Pembayaran ==");
        System.out.println(wallet.getName() + " berhasil?
" + checkout.checkout(cart, regular, wallet));
        System.out.println("Sisa saldo: Rp" +
wallet.getBalance());
        System.out.println(transfer.getName() + "
berhasil? " + checkout.checkout(cart, express,
transfer));

        CreditCardPayment card = new
CreditCardPayment("Siti Rahma", 1_000_000);
        System.out.println(card.getName() + " berhasil? "
+ checkout.checkout(cart, express, card));
        System.out.println("Cicilan 3 bulan: Rp" +
card.monthlyInstallment(checkout.total(cart, express),
3));

        System.out.println("\n== Refund e-book ==");
        System.out.println("Refund e-wallet berhasil? " +
checkout.refund(wallet, ebook.getFinalPrice()));
        System.out.println("Saldo setelah refund: Rp" +
wallet.getBalance());

        // checkout.refund(transfer, 1_000); // compile
error: BankTransferPayment is not RefundablePayment

        // L5.c: Static nested, inner, dan anonymous
class
```

```
        Address address = new Address("Jl. Moh Hatta No.
10", "Padang", "25121");
        Order order = new Order("ORD-0001", address);
        order.addLine(coffee, 2);
        order.addLine(ebook, 1);

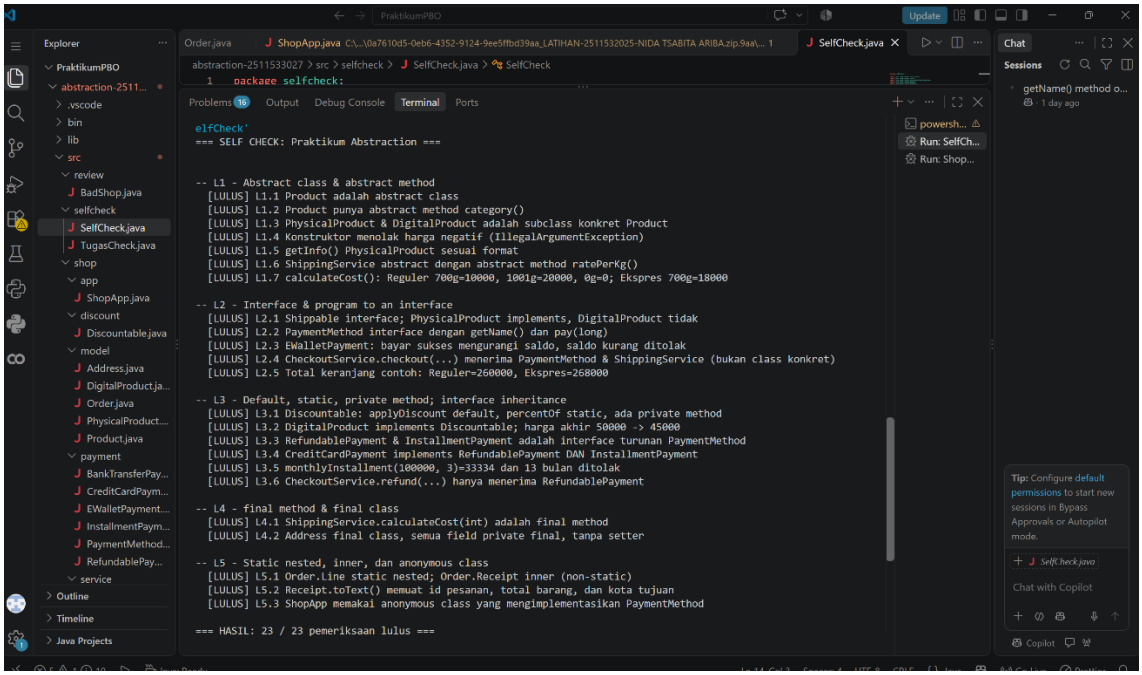
        Order.Receipt receipt = order.new Receipt(); //
inner object needs an outer object
        System.out.println("\n" + receipt.toText());

        PaymentMethod voucher = new PaymentMethod() { //
anonymous class
            private long remaining = 50_000;
            @Override
            public String getName() { return "Voucher
Promo Hari Ini"; }
            @Override
            public boolean pay(long amount) {
                if (amount <= 0 || amount > remaining) {
                    return false;
                }
                remaining -= amount;
                return true;
            }
        };

        System.out.println("\n" + voucher.getName() + "
untuk e-book berhasil? "
            + checkout.checkout(List.of(ebook), regular,
voucher));
    }
}
```

Menambahkan anonymous class. Anonymous class digunakan ketika suatu interface (dalam hal ini PaymentMethod) ingin diimplementasikan secara langsung di dalam kode (inline) hanya untuk satu kali pemakaian tanpa perlu mendeklarasikan file class terpisah. Di dalam ShopApp, anonymous class diterapkan untuk mensimulasikan objek voucher promo hari ini yang memiliki batasan sisa saldo.

2.1.6 SelfCheck



2.1.7 Pesan Error Compiler

a. L1

Uji coba kode: Product p = new Product("X", "X", 1000);

Error

Karena class Product dideklarasikan sebagai abstract class. Tidak boleh membuat objek dengan new pada class abstract karena fungsinya sebagai cetak biru induk bagi subclass nya.

b. L3

Tulis kode: checkout.refund(new BankTransferPayment("Bank Nagari"), 1_000); di ShopApp.

Error

Karena method refund() pada CheckoutService hanya menerima parameter yang mengimplementasikan interface RefundablePayment. Class

BankTransferPayment tidak mengimplementasikan RefundablePayment, sehingga kompiler menolak kode tersebut sebelum program dijalankan.

c. **L4**

Kode: @Override

public long calculateCost(int weightGrams) { return 0; }

Error

Karena method calculateCost() pada ShippingService telah diberi modifier final. Kata kunci final mengunci implementasi method tersebut agar tidak dapat di-override atau diubah rumusnya oleh subclass mana pun.

2.1.8 Keluaran ShopApp

```
PS D:\PraktikumPBO\Praktikum6\Latihan\abstraction-2511533027>  & 'C:\Users\hp\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D:\PraktikumPBO\Praktikum6\Latihan\abstraction-2511533027\bin' 'shop.app.ShopApp'
== Katalog ==
[Fisik] Kopi Arabika Gayo 250g - Rp85000
[Fisik] Rendang Kemasan 250g - Rp120000
[Digital] E-book Resep Kue Nusantara - Rp45000

== Ringkasan ==
Subtotal      : Rp250000
Berat kiriman : 700 g
Total Reguler : Rp260000
Total Ekspres : Rp268000

== Pembayaran ==
E-Wallet Dompetku berhasil? true
Sisa saldo: Rp40000
Transfer Bank Nagari berhasil? true
Kartu Kredit (Siti Rahma) berhasil? true
Cicilan 3 bulan: Rp89334

== Refund e-book ==
Refund e-wallet berhasil? true
Saldo setelah refund: Rp85000

STRUK PESANAN ORD-0001
- Kopi Arabika Gayo 250g x2 = Rp170000
- E-book Resep Kue Nusantara x1 = Rp45000
Total barang: Rp215000
Kirim ke: Jl. Moh Hatta No. 10, Padang 25121

Voucher Promo Hari Ini untuk e-book berhasil? true
PS D:\PraktikumPBO\Praktikum6\Latihan\abstraction-2511533027>
```

2.2 Tugas Praktikum

2.2.1 E.0 Parameter pribadi berbasis NIM

```
PS D:\PraktikumPBO\Praktikum6\Latihan\abstraction-2511533027> java -cp bin tools.NimParams 2511533027
Parameter tugas untuk NIM 2511533027
  CARD_ADMIN_PERCENT  = 2
  TRANSFER_ADMIN_FEE  = 3500
  SMS_MAX_LENGTH      = 127
PS D:\PraktikumPBO\Praktikum6\Latihan\abstraction-2511533027>
```

2.2.2 T1 – Evolusi API dan konflik diamond

a. **T1.a – Cara yang salah dulu**

```java
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    long adminFee(long amount);

    /** @return true if the payment success. */
    boolean pay(long amount);
}
```

Ada 4 buah error, di Class sebagai berikut:

- EWalletPayment
- BankTransferPayment
- CreditCardPayment
- Anonymous class di voucher di ShopApp

Karena anonymous class voucher mengimplementasikan PaymentMethod secara langsung. Saat method abstract baru ditambahkan ke PaymentMethod, seluruh konkret maupun anonymous class yang mengimplementasikan interface tersebut wajib memberikan override/implementasi untuk method baru tersebut. Jika tidak, compiler akan menolak kode karena kelas dianggap tidak memenuhi kontrak interface.

b. T1.b – Default method

```java
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    default long adminFee(long amount) {
        return 0;
    }

    /** @return true if the payment success. */
    boolean pay(long amount);
}
```

c. T1.c – Override sesuai kebutuhan

CreditCardPayment

```java
@Override
    public long adminFee(long amount) {
```

```
        return  amount *
shop.StudentInfo.CARD_ADMIN_PERCENT / 100;
    }
```

BankTransferPayment

```
@Override
    public long adminFee(long amount) {
        return shop.StudentInfo.TRANSFER_ADMIN_FEE;
    }
```

EWalletPayment: Tidak perlu di-override (otomatis mengadopsi nilai 0 dari default method).

d. T1.d – CheckoutService

```
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment; // <--- Jangan
lupa import ini!
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {
    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }
```

```
    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) {
// a capability, not a class
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items,
ShippingService shipping) {
        return subtotal(items) +
shipping.calculateCost(totalWeight(items));
    }

    public boolean checkout(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
            return false;
        }
        long finalAmount = grandTotal(items, shipping,
payment);
        return payment.pay(finalAmount);
    }

    public boolean refund(RefundablePayment payment, long
amount) {
        return payment.refund(amount);
    }
```

```
    public long grandTotal(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        long baseTotal = total(items, shipping);
        return baseTotal + payment.adminFee(baseTotal);
    }

}
```

Pembaruan agar dapat menghitung total keseluruhan biaya belanja yang sudah mencakup biaya admin (admin fee) dari masing-masing metode pembayaran, serta memastikan proses checkout berjalan dengan memperhitungkan biaya admin tersebut.

e. T1.e – Konflik diamond

```
package shop.payment;

public interface RefundablePayment extends PaymentMethod
{
    boolean refund(long amount);
    default String policy() {
        return "Refund maksimal 7 hari";
    }

}
package shop.payment;

public interface InstallmentPayment extends PaymentMethod
{
    int maxInstallmentMonths();

    /** Monthly amount, rounded up. */
    default long monthlyInstallment(long amount, int
months) {
        if (months < 1 || months >
maxInstallmentMonths()) {
```

```
            throw new IllegalArgumentException(
                "Months must be between 1 and " +
maxInstallmentMonths());
        }
        return (amount + months - 1) / months;
    }

    default String policy() {
        return "Cicilan maksimal " +
maxInstallmentMonths() + " bulan";
    }
}
```

1) Mengapa EWalletPayment tidak perlu meng-override policy()?

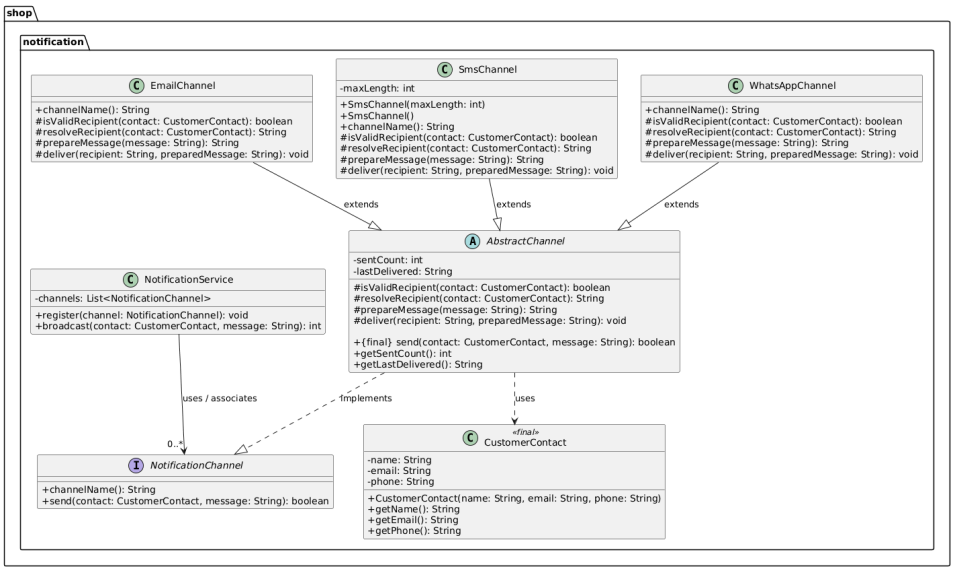
   Karena EWalletPayment hanya mengimplementasikan satu interface yang memiliki method policy(), yaitu RefundablePayment (lewat interface inheritance).

2) Mengapa Java tidak memilih salah satu default secara otomatis?

   Java mengutamakan kejelasan dan keamanan dalam penulisan kode. Oleh karena itu, pengembang harus menentukan sendiri cara menyelesaikan konflik agar program tidak menjalankan fungsi yang salah saat digunakan.

2.2.3 T2 – Merancang sistem notifikasi pesanan

a. Langkah 1



b. **Langkah 2**

AbstractChannel

```
package shop.notification;

public abstract class AbstractChannel implements
NotificationChannel {
    private int sentCount = 0;
    private String lastDelivered = "";

    protected abstract boolean
isValidRecipient(CustomerContact contact);
    protected abstract String
resolveRecipient(CustomerContact contact);
    protected abstract String prepareMessage(String
message);
    protected abstract void deliver(String recipient,
String preparedMessage);

    @Override
    public final boolean send(CustomerContact contact,
String message) {
        if (message == null || message.isBlank()) {
            return false;
        }

        if (!isValidRecipient(contact)) {
            return false;
        }

        String recipient = resolveRecipient(contact);
        String prepared = prepareMessage(message);
```

```
        if (recipient == null || recipient.isBlank() ||
prepared == null || prepared.isBlank()) {
            return false;
        }

        deliver(recipient, prepared);
        lastDelivered = prepared;
        sentCount++;
        return true;
    }

    public int getSentCount() {
        return sentCount;
    }
    public String getLastDelivered() {
        return lastDelivered;
    }

}
```

CustomerContact

```
package shop.notification;

public final class CustomerContact {
    private final String name;
    private final String email;
    private final String phone;

    public CustomerContact(String name, String email,
String phone) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must
not be blank");
```

```
        }

        this.name = name;
        this.email = (email == null) ? "" : email;
        this.phone = (phone == null) ? "" : phone;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getPhone() { return phone; }

}
```

EmailChannel

```
package shop.notification;

public class EmailChannel extends AbstractChannel {
    @Override
    public String channelName() {
        return "Email";
    }

    @Override
    protected boolean isValidRecipient(CustomerContact
contact) {
        return contact.getEmail() != null &&
contact.getEmail().contains("@");
    }

    @Override
    protected String resolveRecipient(CustomerContact
contact) {
        return contact.getEmail();
```

```
    }

    @Override
    protected String prepareMessage(String message) {
        return message;
    }

    @Override
    protected void deliver(String recipient, String
preparedMessage) {
        System.out.println("[Email to " + recipient + "]
" + preparedMessage);
    }

}
```

NotificationChannel

```
package shop.notification;

public interface NotificationChannel {
    String channelName();
    boolean send(CustomerContact contact, String
message);
}
```

NotificationService

```
package shop.notification;

import java.util.ArrayList;
import java.util.List;

public class NotificationService {
```

```
    private final List<NotificationChannel> channels =
new ArrayList<>();
    public void register(NotificationChannel channel) {
        if (channel != null) {
            channels.add(channel);
        }
    }

    public int broadcast(CustomerContact contact, String
message) {
        int sentCount = 0;
        for (NotificationChannel channel : channels) {
            if (channel.send(contact, message)) {
                sentCount++;
            }
        }
        return sentCount;
    }

}
```

SmsChannel

```
package shop.notification;

import shop.StudentInfo;

public class SmsChannel extends AbstractChannel {
    private final int maxLength;

    public SmsChannel(int maxLength) {
        if (maxLength < 3) {
            throw new IllegalArgumentException("Panjang
SMS minimal 3 karakter");
```

```
        }
        this.maxLength = maxLength;
    }

    public SmsChannel() {
        this.maxLength = StudentInfo.SMS_MAX_LENGTH;
    }

    @Override
    public String channelName() {
        return "SMS";
    }

    @Override
    protected boolean isValidRecipient(CustomerContact
contact) {
        return contact.getPhone() != null &&
contact.getPhone().startsWith("08");
    }

    @Override
    protected String resolveRecipient(CustomerContact
contact) {
        return contact.getPhone();
    }

    @Override
    protected String prepareMessage(String message) {
    if (message.length() > maxLength) {
        return message.substring(0, maxLength - 3) +
"...";
    }
    return message;
```

```
    }

    @Override
    protected void deliver(String recipient, String
preparedMessage) {
        System.out.println("[SMS to " + recipient + "] "
+ preparedMessage);
    }
}
```

WhatsAppChannel

```
package shop.notification;

public class WhatsAppChannel extends AbstractChannel {
    @Override
    public String channelName() {
        return "WA";
    }

    @Override
    protected boolean isValidRecipient(CustomerContact
contact) {
        String phone = contact.getPhone();
        if (phone == null || phone.length() < 10)
            return false;
        return phone.startsWith("08") ||
phone.startsWith("+62");
    }

    @Override
    protected String resolveRecipient(CustomerContact
contact) {
        return contact.getPhone();
```

```
    }

    @Override
    protected String prepareMessage(String message) {
        return "[Toko UMKM] " + message;
    }

    @Override
    protected void deliver(String recipient, String
preparedMessage) {
        System.out.println("[WA to " + recipient + "] " +
preparedMessage);
    }

}
```

c. **Langkah 3**

```
package shop.app;

import java.util.List;
import shop.model.DigitalProduct;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.notification.CustomerContact;
import shop.notification.EmailChannel;
import shop.notification.NotificationChannel;
import shop.notification.NotificationService;
import shop.notification.SmsChannel;
import shop.notification.WhatsAppChannel;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.EWalletPayment;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment;
```

```java
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.ShippingService;
import shop.StudentInfo;

public class TugasApp {
    public static void main(String[] args) {
        Product coffee = new PhysicalProduct("KOPI-001",
"Kopi Arabika Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-002",
"Rendang Kemasan 250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003", "E-
book Resep Kue Nusantara", 50_000, "https://toko-
umkm.example/unduh/ebk-003");
        List<Product> cart = List.of(coffee, rendang,
ebook);

        CheckoutService checkout = new CheckoutService();
        ShippingService express = new ExpressShipping();

        PaymentMethod wallet = new
EWalletPayment("Dompetku", 300_000);
        PaymentMethod transfer = new
BankTransferPayment("Bank Nagari");
        CreditCardPayment card = new
CreditCardPayment("Siti Rahma", 1_000_000);

        System.out.println("Tugas oleh " +
StudentInfo.NAME + " (" + StudentInfo.NIM + ")");
        System.out.println("== T1: Total dengan biaya
admin (Ekspres) ==");
```

```
        System.out.println(wallet.getName() + " Rp" +
checkout.grandTotal(cart, express, wallet) + " (admin Rp"
+ wallet.adminFee(checkout.total(cart, express)) + ")");
        System.out.println(transfer.getName() + " Rp" +
checkout.grandTotal(cart, express, transfer) + " (admin
Rp" + transfer.adminFee(checkout.total(cart, express)) +
")");
        System.out.println(card.getName() + ": Rp" +
checkout.grandTotal(cart, express, card) + " (admin Rp" +
card.adminFee(checkout.total(cart, express)) + ")");

        System.out.println("Kebijakan kartu kredit : " +
card.policy());
        if (wallet instanceof RefundablePayment
refWallet) {
            System.out.println("Kebijakan e-wallet     :
" + refWallet.policy());
        }

        System.out.println("\n== T2: Notifikasi status
pesanan ==");
        NotificationService notificationService = new
NotificationService();
        notificationService.register(new EmailChannel());
        notificationService.register(new
SmsChannel(StudentInfo.SMS_MAX_LENGTH));
        notificationService.register(new
WhatsAppChannel());

        notificationService.register(new
NotificationChannel() {
            @Override
            public String channelName() {
```

```
                return "LOG";
            }

            @Override
            public boolean send(CustomerContact contact,
String message) {
                System.out.println("[LOG] " +
contact.getName() + ": " + message);
                return true;
            }
        });

        String longMessage = "Pesanan ORD-0001 sudah
dikirim dengan kurir Ekspres. " +
                "Nomor resi akan muncul di aplikasi dalam
1x24 jam. " +
                "Terima kasih telah berbelanja produk
UMKM Sumatera Barat di toko kami!";

        CustomerContact contact1 = new
CustomerContact("Siti Rahma", "siti@example.com",
"081234567890");
        int sent1 =
notificationService.broadcast(contact1, longMessage);
        System.out.println("Terkirim ke Siti lewat " +
sent1 + " kanal\n");

        CustomerContact contact2 = new
CustomerContact("Budi", "budi@invalid",
"+6281298765432");
        int sent2 =
notificationService.broadcast(contact2, longMessage);
```

```
        System.out.println("Terkirim ke Budi lewat " +
sent2 + " kanal");
    }
}
```

d. Pertanyaan

1. Mengapa ada interface dan abstract class, bukan salah satunya saja?

   NotificationChannel digunakan sebagai aturan dasar yang harus dimiliki setiap kanal notifikasi, sedangkan AbstractChannel menyediakan kode umum seperti validasi pesan, proses pengiriman, dan pencatatan jumlah pesan. Dengan begitu, setiap kanal tidak perlu menulis ulang proses yang sama.

2. Apa akibatnya jika send(...) tidak dibuat final?

   Jika tidak dibuat final, class turunannya bisa mengubah isi method send(). Hal ini dapat menyebabkan proses validasi terlewat atau jumlah pesan yang terkirim tidak tercatat dengan benar. Oleh karena itu, final digunakan agar proses pengiriman tetap berjalan sesuai aturan.

3. Mengapa NotificationService menyimpan List, bukan List?

   Karena dengan menggunakan List<NotificationChannel>, NotificationService dapat menerima berbagai jenis kanal notifikasi yang menerapkan interface tersebut, termasuk kanal yang dibuat menggunakan anonymous class. Dengan demikian, program menjadi lebih fleksibel dan mudah dikembangkan.

2.2.4 T3 – Code review BadShop

a. Masalah

No.	Lokasi (class/method)	Masalah	Dampak nyata (bug atau risiko)	Perbaikan yang diusulkan
1.	ExpressShipping.calculateCost()	Meng-override method perhitungan biaya padahal ShippingService tidak mengunci	Terjadi error di mana tarif pengiriman bernilai 0 karena perhitungan salah	Buat calculateCost() bertipe final di ShippingService dan paksa

		rumusnya.	di-override secara manual.	subclass hanya meng-override tarif ratePerKg().
2.	Address / Order	Kelas Address memiliki setter dan bersifat mutable.	Alamat ORD-1 berubah sendiri ketika alamat ORD-2 diubah karena referensi objek yang sama digunakan kembali.	Jadikan Address sebagai public final class dengan semua field private final dan hapus semua method setter (Immutable).
3.	GiftVoucher implements Shippable	Interface Shippable dipaksakan pada produk voucher digital yang tidak punya fisik.	Terjadi UnsupportedOperationException saat sistem mencoba mengirim voucher via kurir.	Pisahkan hierarki. GiftVoucher hanya mengimplementasikan PaymentMethod / Discountable, bukan Shippable.
4.	Product	Menggunakan kelas biasa, bukan abstract class.	Dapat melakukan instansiasi langsung new Product() yang tidak valid secara logika bisnis.	Deklarasikan Product sebagai public abstract class dengan abstract String category().
5.	Order.Line	Deklarasi kelas	Setiap objek Line	Ubah deklarasi

		nested tidak menggunakan kata kunci static.	menyimpan referensi tersembunyi (implicit reference) ke objek luar Order sehingga boros memori dan tidak bisa dibuat mandiri.	menjadi public static class Line di dalam Order.

b. Refactor

```
package review.fixed;

import java.util.ArrayList;
import java.util.List;

public class BadShopRefactored {
    public interface PaymentMethod {
        boolean pay(long amount);
        String printInvoice();
    }

    public interface Shippable {
        long calculateCost(int weightGrams);
    }

    public static class GiftVoucher implements
PaymentMethod {
        private double balance = 100000.0;

        @Override
        public boolean pay(long amount) {
```

```
            if (amount <= 0 || amount > balance) {
                return false;
            }
            balance -= amount;
            return true;
        }

        @Override
        public String printInvoice() {
            return "Voucher, remaining balance " +
balance;
        }

        public double getBalance() {
            return balance;
        }
    }

    public static abstract class ShippingService
implements Shippable {
        private final String name;

        public ShippingService(String name) {
            this.name = name;
        }

        public String getName() {
            return name;
        }

        protected abstract long ratePerKg();

        @Override
```

```
        public final long calculateCost(int weightGrams)
{
            if (weightGrams <= 0) return 0;
            long kilograms = (weightGrams + 999L) / 1000;
            return kilograms * ratePerKg();
        }
    }

    public static class ExpressShipping extends
ShippingService {
        public ExpressShipping() {
            super("Ekspres");
        }

        @Override
        protected long ratePerKg() {
            return 18000;
        }
    }

    public static class Cart {
        private final List<Item> items = new
ArrayList<>();
        public static class Item {
            private final String name;
            private final long price;
            private final int quantity;

            public Item(String name, long price, int
quantity) {
                this.name = name;
                this.price = price;
                this.quantity = quantity;
```

```
        }

        public long subtotal() {
            return price * quantity;
        }

        public String getName() {
            return name;
        }
    }

    public void add(Item item) {
        items.add(item);
    }

    public long total() {
        long total = 0;
        for (Item item : items) {
            total += item.subtotal();
        }
        return total;
    }
}

public static final class Address {
    private final String street;
    private final String city;

    public Address(String street, String city) {
        this.street = street;
        this.city = city;
    }
```

```
        public String getStreet() { return street; }
        public String getCity() { return city; }

        @Override
        public String toString() {
            return street + ", " + city;
        }
    }

    public static class SimpleOrder {
        private final String id;
        private final Address address;

        public SimpleOrder(String id, Address address) {
            this.id = id;
            this.address = address;
        }

        public Address getAddress() {
            return address;
        }

        @Override
        public String toString() {
            return id + " -> " + address;
        }
    }

    public interface Discount {
        long apply(long price);
    }
```

```
    public static class FlashSaleDiscount implements
Discount {
        private final int percent;

        public FlashSaleDiscount(int percent) {
            this.percent = percent;
        }

        @Override
        public long apply(long price) {
            return price - (price * percent / 100);
        }
    }

    public static void main(String[] args) {
        Cart cart = new Cart();
        cart.add(new Cart.Item("Kopi Gayo", 85_000, 2));
        System.out.println("Total cart: " +
cart.total());

        ShippingService express = new ExpressShipping();
        System.out.println("Ongkir " + express.getName()
+ " 2 kg: " + express.calculateCost(2000));

        Address home1 = new Address("Jl. Sudirman 10",
"Padang");
        Address home2 = new Address("Jl. Sudirman 10",
"Bukittinggi");
        SimpleOrder first = new SimpleOrder("ORD-1",
home1);
        SimpleOrder second = new SimpleOrder("ORD-2",
home2);
        System.out.println(first);
```

```
        System.out.println(second);

        Discount flashSale = new FlashSaleDiscount(30);
        System.out.println("Harga flash sale: " +
flashSale.apply(100_000));

        GiftVoucher voucher = new GiftVoucher();
        boolean paid = voucher.pay(30_000);
        System.out.println(paid + " | sisa Rp" +
(long)voucher.getBalance());
    }

}
```

Telah diimplementasikan pada paket review.fixed di mana bug ongkir 0 berhasil diatasi, mutasi alamat tercegah, serta exception akibat pengiriman voucher dapat terhindar.

2.2.5 T4 – Refleksi: kapan memakai apa

a. Tabel 1 – Abstract class vs interface

Pertanyaan desain	Abstract class atau interface?	Alasan	Contoh dari proyek
Subclass berbagi field dan konstruktor	Abstract Class	Abstract class memungkinkan pewarisan private final dan logika inisialisasi constructor bersama.	Product (membagi SKU, nama, harga) dan ShippingService (membagi nama kurir).
Satu class perlu beberapa	Interface	Java tidak bisa meng multiple inheritance	CreditCardPayment implements RefundablePayment, InstallmentPayment

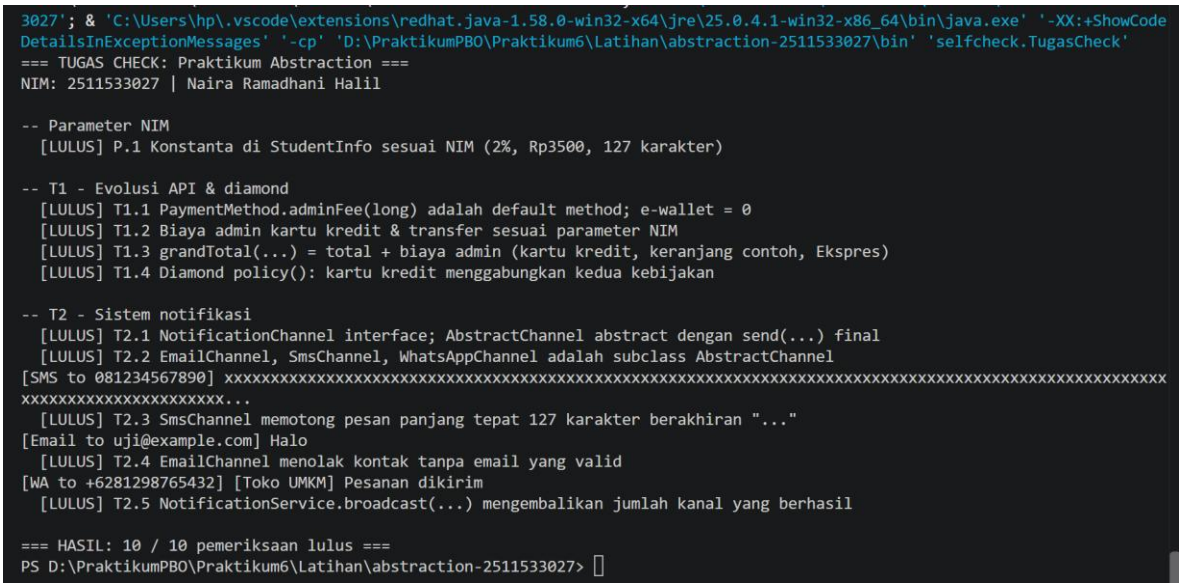
kemampuan sekaligus		pada kelas, tapi satu kelas bisa meng implements banyak interface.	
Rumus bersama yang tidak boleh diubah subclass	Abstract Class	Perpaduan abstract class dan final method bisa mengunci algoritma utama sambil menyerahkan detail ke subclass.	ShippingService.calculateCost() (rumus pembulatan berat dikunci dengan final).
Kontrak yang perlu ditambah method baru di kemudian hari	Interface	Menggunakan default method memungkinkan penambahan fitur baru pada interface tanpa merusak kode lama.	Penambahan adminFee() dan policy() pada PaymentMethod.

b. Tabel 2 – Jenis nested class

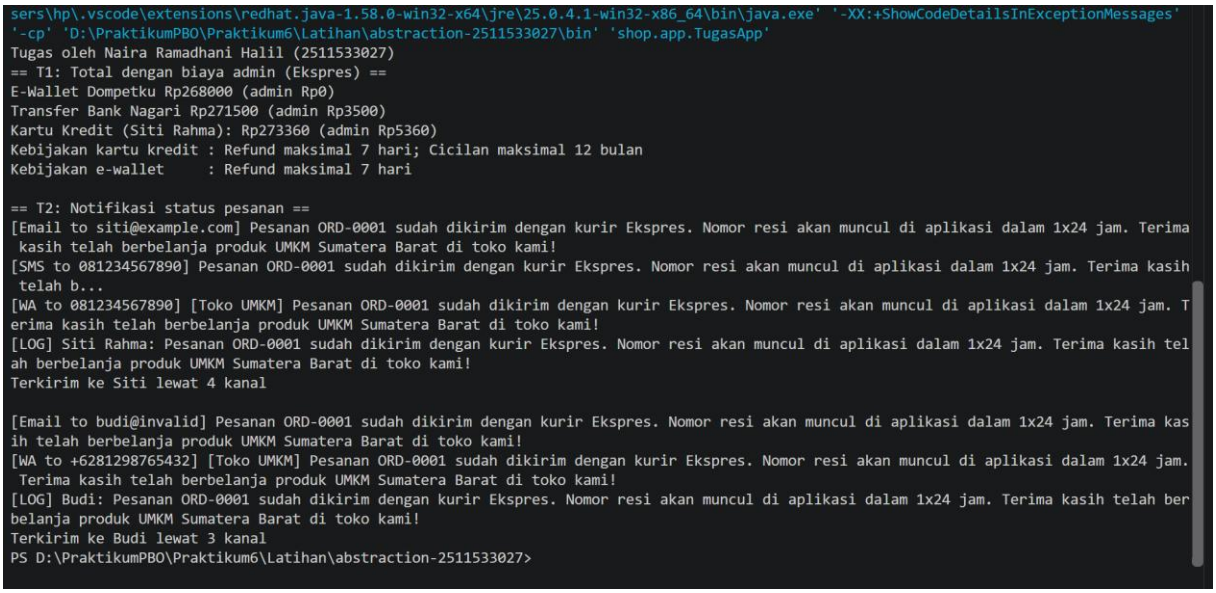
Kondisi	Static nested, inner, atau anonymous?	Alasan	Contoh dari proyek
Class pembantu tidak membaca data objek luar	Static Nested Class	Tidak perlu referensi sembunyi ke objek luar, hemat	Order.Line (hanya menyimpan produk dan

		memori dan berdiri sendiri.	jumlah).
Objek harus membaca field objek luarnya	Inner Class	Terikat langsung ke instance objek luar dan dapat membaca.	Order.Receipt (membaca orderId dan shippingAddress milik Order).
Implementasi sekali pakai, tidak perlu nama	Anonymous Class	Praktis untuk implementasi cepat tanpa membuat file kelas sendiri yang jarang dipakai ulang.	Implementation PaymentMethod voucher promo pada ShopApp / TugasApp.

2.2.6 TugasCheck


3027'; & 'C:\Users\hp\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCode
DetailsInExceptionMessages' '-cp' 'D:\PraktikumPBO\Praktikum6\Latihan\abstraction-2511533027\bin' 'selfcheck.TugasCheck'
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511533027 | Naira Ramadhani Halil
-- Parameter NIM
[LULUS] P.1 Konstanta di StudentInfo sesuai NIM (2%, Rp3500, 127 karakter)
-- T1 - Evolusi API & diamond
[LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
[LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
[LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
[LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
-- T2 - Sistem notifikasi
[LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
[LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
[SMS to 081234567890] xxxx

2.2.7 Keluaran TugasApp



BAB III
PENUTUP

3.1 Kesimpulan

Praktikum ini memberikan pemahaman tentang penerapan berbagai konsep abstraksi Java melalui proyek sistem toko online UMKM. Penggunaan abstract class (Product dan ShippingService) membantu membuat kerangka dasar yang dapat digunakan oleh class turunannya. Sementara itu, interface (Shippable, PaymentMethod, dan Discountable) digunakan untuk mengatur kemampuan yang harus dimiliki oleh suatu class sehingga program lebih fleksibel.

Selain itu, praktikum ini juga mempelajari penggunaan default method dan static method pada interface untuk menambahkan fungsi baru tanpa harus mengubah seluruh class yang sudah ada. Konsep interface inheritance dan penyelesaian konflik pewarisan juga diterapkan agar hubungan antarinterface tetap teratur. Penggunaan kata kunci final serta berbagai jenis nested class membantu menjaga agar kode lebih aman, terstruktur, dan mudah dikembangkan.